Modeling Many Worlds: Rule-Conditioned Neural Cellular Automata for Simulating Puzzle Games

Anonymous Author(s)
Affiliation
Address
email

Abstract

World models, in which the dynamics of a simulated environment are distilled into a neural substrate, have shown promise as differentiable surrogates for sample-efficient policy learning and gradient-based planning. So far, these models have largely been trained on individual games with visually rich pixel output but relatively simple or uniform underlying mechanics. We train a rule-conditioned world model on a dataset of hundreds of mechanically complex and diverse human-authored grid-based puzzle games. Foregoing pixels, we model the full discrete state of the environment at each timestep, and use Neural Cellular Automaton (NCA) to mimic the original game engine's repeated application of atomic local update rules. This allows us to model environment dynamics with near-perfect fidelity over long time horizons. To support arbitrary game rules, the NCA attends to the latents of a jointly-optimized auto-encoder of symbolic game descriptions. This allows us to sample novel latent games that can be decoded into symbolic programs, and to fit out-of-distribution environment transitions to game code by optimizing the unobserved latent game embedding through the frozen NCA dynamics model. By scaling data in such a regime, we shift from modeling a game to modeling a game engine, or a space of many possible worlds organized according to their dynamics.

1 Introduction

Learned world models have been studied across pixel-based video-game benchmarks (Atari, ALE, MuJoCo image observations) and procedurally generated environments. The standard recipe pairs a high-capacity encoder–decoder—typically a CNN, U-Net, or recently a video Transformer—with an action-conditioned dynamics module that predicts the next observation given the current observation and an action. These architectures excel when the underlying state is high-dimensional and the dynamics are smooth in pixel space.

PuzzleScript games are a different regime. Each game is fully specified by a short text file describing object kinds, layered grid levels, and a list of declarative rules of the form `[ Player | Crate ] -> [ | Player Crate ]`. The engine evaluates these rules in order on the grid at every tick, with `again` rules causing the rule list to be re-evaluated until the state stops changing. The state is therefore a *discrete multi-channel grid*, the dynamics are *deterministic and combinatorial*, and the "hard" computation per tick is a sequence of small local updates applied repeatedly. Pixel-style world models do not match this substrate well: they spend capacity modeling continuous-valued tile appearance, lack any inductive bias toward local rule application, and cannot easily share parameters across the engine's outer `again` loop.

We propose a different architecture: a *rule-conditioned Neural Cellular Automaton* (NCA) world model. A perceiver-style encoder maps the tokenized game spec to K slot vectors representing rule

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

embeddings; the dynamics body is a single local update layer applied for T iterations on a hidden grid, with cells cross-attending to those slots at every step. The same body parameters are reused across iterations, mirroring the engine's outer loop, while the slot encoder provides per-game conditioning that makes the model game-agnostic.

Contributions.

1. A rule-conditioned NCA world model that learns multi-game dynamics for PuzzleScript with a single shared local update rule, conditioned via cross-attention on perceiver-style slots encoding the game's rule set.
2. A controlled comparison against non-NCA baselines (CNN, U-Net, ViT) that share the rule-slot encoder but differ in the spatial-update body. The comparison isolates the contribution of the iterated-local-update inductive bias.
3. Empirical analyses of three architectural axes that the NCA couples by design but which can be ablated independently: global pooling features (axial / cumulative max / global max), weight sharing across NCA steps (a hierarchical $n_{\text{layers}} \times n_{\text{repeats}}$ factorization), and adaptive halting (PonderNet-style learned halt vs. convergence-based stopping).
4. A characterization of an architectural ceiling on the multi-grid synthetic-data regime, where the model loses the per-state memorization shortcut and is forced to learn transferable rule application. Our experiments suggest current rule-conditioned NCAs do not reliably escape this ceiling without additional supervision; we discuss candidate interventions.

Why baselines from a different family. Reviewers of NCA papers often (rightly) ask whether the same task could be solved by an off-the-shelf grid-to-grid architecture without the recurrence. Our baselines are designed to answer exactly that: each shares the rule-slot encoder verbatim with our model, so the comparison isolates the spatial-update body, not the conditioner. The CNN baseline tests whether iteration is necessary at all; the U-Net baseline tests whether multi-scale hierarchy with skips is a better substitute for repeated local updates; the ViT baseline tests whether unrestricted global attention beats locality plus iteration.

2 Related Work

Neural Cellular Automata. Neural Cellular Automata learn a local update rule that, applied repeatedly to a grid of hidden states, gives rise to globally coordinated behavior [7]. Subsequent work has demonstrated NCAs that grow morphologies from a seed, classify under distribution shift, perform texture synthesis, and self-organize attention [9, 10, 8]. NCAs are typically trained with stochastic update schedules and a "pool" of intermediate states for stability. Recent work treats the NCA's iteration count as a meaningful axis of computation and studies when more iterations buy more reasoning. Our use of NCAs as *action-conditioned world models* extends this line: rather than asking the NCA to converge to a static target, we ask it to predict the engine's deterministic next state at every tick.

World models and self-prediction. World-model literature dates back at least to Schmidhuber's curiosity work and was popularized for vision-based RL by Ha and Schmidhuber [4]. Recent transformer-based world models [6, 11] compress images to discrete tokens and roll out dynamics autoregressively over those tokens; PWM-style methods integrate planning into a learned model. Our problem domain—fully discrete grid observations, authored rule sets, dozens of related games sharing a substrate—is closer to procedural-content benchmarks like NetHack and CHAI than to Atari. Compared to MuZero-style models that learn an abstract latent dynamics, our model commits to predicting the engine's next state in its native grid representation, which makes ground-truth supervision exact and lets us compare to the engine cell-by-cell.

PuzzleScript and rule-based puzzle games. PuzzleScript [5] is a domain-specific language for grid-based puzzle games that supports a rich family of authoring patterns: chained pushing (`[ Crate | Crate ] -> [ Crate | Crate ]`), `again`-driven simulation (gravity, ball trajectories, light beams), multi-character control, and rule priorities. Several hundred community games are publicly available and share both a tokenization scheme (the engine source) and a common state

encoding, making the family a natural multi-game benchmark for world models. Prior work has studied PuzzleScript primarily for procedural content generation and human play; to our knowledge, ours is the first systematic study of learned forward models on a broad PuzzleScript gallery.

Adaptive computation. PonderNet [1] learns a per-input halting distribution over a recurrent module, with a KL prior over the number of iterations. Adaptive Computation Time [3] and Universal Transformers [2] are predecessors. Our adaptive-halt experiments adapt PonderNet to NCA world models with a shared body, so that "halt at step k" coherently asks the same rule set to converge in k iterations rather than selecting between k different models.

Learned local-update grids more broadly. ConvLSTMs [12], graph neural networks, and diffusion-style iterative refinement all share with NCAs the high-level idea of repeated local computation. NCAs differ in committing to a *shared* (across iterations) update rule on a fixed grid, which matches the PuzzleScript engine's outer-loop semantics. A detailed comparison to procedural / engine-aligned NCA work is deferred to the appendix.

3 Method

3.1 Problem setup

A PuzzleScript game is a tuple $(\mathcal{O}, R, \{L_i\})$ where $\mathcal{O}$ is an ordered list of object kinds, R is a list of rewrite rules, and $\{L_i\}$ are authored levels. Each level is a multi-channel grid $s \in \{0,1\}^{C\times H\times W}$ where channel c is the indicator of object kind c at each cell. An action $a \in \{\texttt{up}, \texttt{down}, \texttt{left}, \texttt{right}, \texttt{action}\}$ together with the current state s_t deterministically yields a next state s_{t+1} via the engine. The state can also carry a terminal *won* bit, since some rules trigger a win on certain conditions.

We train a model $f_\theta(s_t, a_t, \mathrm{spec}) \mapsto \widehat{s}_{t+1}$ where spec is the tokenized game source (a sequence of integers from a shared $\sim$140-symbol vocabulary that covers the engine's grammar plus per-game object names and 5×5 sprite RGBA values when sprite tokens are enabled). At evaluation we roll the model out autoregressively and compare cell-by-cell against the engine.

3.2 Rule-conditioned NCA world model

Slot encoder. A perceiver-style encoder Enc_ϕ maps the game's token sequence $\mathrm{spec} \in \mathbb{Z}^S$ to K slot vectors $z \in \mathbb{R}^{K\times d}$. The encoder applies ℓ self-attention layers over the tokens, then K learned query vectors cross-attend into the contextualized token sequence to produce z. Slots are shared across cells and remain fixed throughout the rollout.

NCA dynamics body. We embed the input (s_t, a_t) into a hidden grid $h^{(0)} \in \mathbb{R}^{H\times W\times d_h}$ via a single dense layer applied to the per-cell concatenation of s_t's channels and a broadcast one-hot action. We then apply T NCA steps:

$$h^{(k+1)} = h^{(k)} + W_o\,\mathrm{GELU}\big(W_c\,[\,h^{(k)}\,;\,h^{(0)}\,]_{3\times3} \;+\; \mathrm{XAttn}(h^{(k)}, z)\big)\,, \tag{1}$$

where $[\,\cdot\,]_{3\times3}$ denotes a 3×3 convolution producing a vector at each cell, XAttn is a multi-head attention with each cell as a query and the K slots as keys/values, and W_o, W_c are shared across all T steps. The *input-skip* concatenation re-injects the embedded (s_t, a_t) at every step, preventing the deep unroll from drifting.

Pool features. A bare 3×3 convolution can only propagate one cell of context per NCA step, but several PuzzleScript rule patterns reference cells that are arbitrarily far apart (`[ X | ... | Y ]` along an axis, or `[X][Y]` anywhere in the grid). We therefore augment each step with three optional pool feature stacks: axis max pool (row-max and col-max), axis cumulative max (directional prefix-max in each of four directions), and global max pool. These features are computed from the current hidden grid and broadcast back to every cell before the convolution, exposing non-local information that would otherwise require $\Omega(\max(H, W))$ NCA steps to surface.

Hierarchical weight sharing. Inspired by the engine's two natural levels of iteration (an inner ordered list of rules + an outer `again` re-evaluation loop), we factor $T = n_\ell \cdot n_r$ where n_ℓ distinct layers form the inner block and n_r outer applications share weights. Setting $n_r = T$ recovers a single shared layer applied T times (matches the engine's outer-loop structure); setting $n_r = 1$ recovers the usual per-step body with T distinct layers.

Heads. A shared `readout` dense layer maps the final $h^{(T)}$ cell-wise to next-state logits $\widehat{s}_{t+1} \in \mathbb{R}^{C\times H\times W}$; a separate dense layer over the spatially average-pooled $h^{(T)}$ produces a binary win logit. Training loss is a per-cell BCE on the next-state with an upweight on cells that *change* between s_t and s_{t+1}, plus a binary cross-entropy on the win head.

3.3 Baselines

We compare against three non-NCA baselines that share the slot encoder Enc_ϕ, the input embedding, and the readout heads, but replace the NCA body. Each baseline consumes $h^{(0)}$ and the slots z and emits a final hidden grid $h^{(T)}$ of the same shape; the comparison isolates the contribution of the spatial-update body.

- **CNN**: a stack of n_b residual blocks, each containing the same conv + optional pool + slot-cross-attn + residual gate as one NCA layer, but with *distinct* weights per block and applied once. Tests whether iteration with weight sharing matters at all.
- **U-Net**: a 2-level encoder–decoder with skip connections and a FiLM bottleneck that reads pooled slots. Tests whether multi-scale hierarchy substitutes for repeated local updates.
- **ViT**: n_v Transformer encoder layers over the flattened cell sequence with learned 2D positional embedding, plus a slot cross-attention and feed-forward block per layer. Tests whether unrestricted global attention beats locality plus iteration.

All baselines accept the same input $(s_t, a_t, \mathrm{spec})$ and produce the same $(\widehat{s}_{t+1}, \widehat{w}_{t+1})$ outputs as the NCA, allowing identical training and evaluation pipelines.

3.4 Adaptive halting (optional)

A fixed T is wasteful for easy transitions and insufficient for hard ones. We add an optional halting head: at each NCA step k, a small dense layer reads the spatially pooled $h^{(k)}$ and emits a halt logit λ_k. The induced halt distribution $p_k = \mathrm{sigmoid}(\lambda_k)\prod_{j<k}(1-\mathrm{sigmoid}(\lambda_j))$ weights the per-step loss $\mathcal{L}_{\text{rec}} = \sum_k p_k \mathcal{L}_k$ plus a KL-to-geometric-prior regularizer $\mathrm{KL}(p \,\|\, \mathrm{Geom}(p_0))$. We additionally study a *convergence-stopping* baseline that halts at inference when consecutive predictions stop changing, with no learned halt head.

4 Experimental setup

4.1 Data

We collect $(s_t, a_t, s_{t+1}, w_{t+1})$ transitions for each game by running the C++ PuzzleScript engine under bounded A* search from each authored level. Each transition is cached on disk (bit-packed for roughly $8\times$ compression) and re-used across runs. For multi-game training we draw batches with one share per game per step (*balanced sampling*), which compensates for per-game dataset-size imbalance.

Synthetic levels. For ablations and held-out evaluations we additionally generate synthetic levels by sampling tile-pattern distributions from each game's authored levels and validating each candidate by running the engine to confirm the level is solvable and exposes a non-trivial state space (more details in the appendix). The validated synthetic recipe (per-game sizes, fallback dynamics, multi-grid widths) is what allowed our model to match the authored-data baseline on the NEKOPUZZLE game without overfitting to grid-size artifacts.

4.2 Game suites

We evaluate on three suites of increasing breadth:

- **Single-game** (MICROBAN, 10 authored levels): classic chained Sokoban pushing on small grids. Used for architecture ablations where dynamics complexity is fully controlled.
- **Single-game / synthetic-multi-grid** (SOKOBAN_BASIC): a controlled stress test in which the model is trained on synthetic levels of multiple grid sizes $\{5\times5, 6\times6, 7\times7, 8\times8\}$ and evaluated on the same. The diversity prevents per-state memorization and forces the model to learn the underlying rule.
- **Multi-game** (SCALING_14): a fourteen-game benchmark including chained pushing, gravity, light beams, multi-character control, and several `again`-driven simulations. Per-game grid sizes range from 4×4 to 32×24.

4.3 Architectures and hyper-parameters

All models share the same input embedding (Dense over $(s_t \,\|\, \mathrm{onehot}(a_t))$) and readout heads. The slot encoder is identical across architectures: K=16 slots, d=64 slot dim, two self-attention layers over tokens, four heads.

The NCA uses T=4 NCA steps with n_r=T (single shared layer applied T times), d_h=256, full pool features, input-skip on, optional padding mask for multi-grid. The CNN baseline uses n_b=4 residual blocks; the U-Net uses 2 levels of down/up sampling with bottleneck FiLM; the ViT uses n_v=4 layers, four heads. We report parameter count alongside every metric so the comparison is honest.

Training: Adam, learning rate 3×10^{-4} with cosine decay to 10^{-7}, gradient clipping at global norm 0.5, batch size 32, *change-loss-weight* 5.0 (per-cell BCE upweighted on cells that change between t and $t+1$), and patience-based early stop on the change-accuracy metric. Each run is replicated with three random seeds.

4.4 Evaluation

We report two complementary metrics:

- **One-step change-error**: the per-cell BCE-thresholded error rate on the cells that change between s_t and s_{t+1}, measured under teacher forcing on the held-out portion of each cache.
- **Autoregressive rollout error**: the same per-cell error averaged over a 50-step rollout from each level's start state, under three policies: random actions, BFS-optimal, and A*-optimal.

Per finding F3 in our architecture report (§5), train loss decouples from autoregressive rollout error in the over-capacity regime; reporting both is essential.

5 Results

5.1 Baselines vs. rule-conditioned NCA

Table 1 reports the head-to-head comparison on MICROBAN (10 authored levels), trained for 10k updates per seed at $d_h = 256$ with the canonical recipe (§4). We measure best train cross-entropy loss, end-of-rollout cell-error rate under three policies (random, BFS-optimal, A*-optimal), and final parameter count. The table reports a single seed; the multi-seed synthetic-data comparison is reported in Table 2.

Three findings on this controlled, memorizable single-game setting: **(i)** the NCA with shared weights wins on BFS rollout ($0.11\% \pm 0.16\%$) at the fewest parameters (1.94M), $2.9\times$ better than the same-depth feed-forward CNN ($0.32\% \pm 0.24\%$) at $2.6\times$ fewer params and $4.3\times$ better than the per-step NCA ($0.47\% \pm 0.66\%$); **(ii)** U-Net and ViT both actively underperform here despite parameter counts comparable to or exceeding NCA-shared — U-Net's 2-level downsampling loses fine-grained player position on small grids, and ViT's global self-attention diverges before the first rollout step (mean first-divergence step under 1), suggesting locality plus iteration is a stronger inductive bias for grid-rule dynamics than unrestricted global attention; **(iii)** best train loss is essentially identical

Table 1: **Baselines vs. rule-conditioned NCA on MICROBAN (authored levels).** Mean $\pm$ std over 3 seeds at 10k updates. Lower is better for loss and cell-error; A* first- divergence step is higher-is-better. Param count in millions; architectures sorted by BFS final cell-error.

Architecture	params (M)	best loss	BFS final-err	A* first-div
NCA (shared, T=4, n_r=T)	**1.94**	0.107 ± 0.002	$\mathbf{0.11\% \pm 0.16\%}$	64
CNN (4-block ResNet)	5.00	0.107 ± 0.002	$0.32\% \pm 0.24\%$	89
NCA (per-step, T=4, n_r=1)	7.36	0.107 ± 0.002	$0.47\% \pm 0.66\%$	35
U-Net (2 levels)	6.07	0.108 ± 0.002	$4.84\% \pm 1.03\%$	14
ViT (4 layers)	3.99	0.107 ± 0.002	$10.96\% \pm 0.50\%$	< 1

Table 2: **Baselines vs. rule-conditioned NCA on SOKOBAN_BASIC (multi-grid synthetic).** Mean $\pm$ std over 3 seeds at 10k updates; widths $\{5\times5, 6\times6, 7\times7, 8\times8\}$ with 256 synthetic levels per width. Memorization is unavailable; the model has to learn the slide rule across grid sizes. Architectures sorted by BFS final cell-error.

Architecture	params (M)	best loss	BFS final-err	A* first-div
NCA (shared, T=4, n_r=T)	**3.07**	0.069 ± 0.003	$\mathbf{4.37\% \pm 2.02\%}$	9.2
CNN (4-block ResNet)	5.00	0.069 ± 0.003	$5.56\% \pm 3.12\%$	7.0
NCA (per-step, T=4, n_r=1)	8.49	0.070 ± 0.003	$5.95\% \pm 3.50\%$	13.8
U-Net (2 levels)	6.07	0.071 ± 0.003	$8.33\% \pm 0.97\%$	0.5
ViT (4 layers)	3.99	0.084 ± 0.002	$12.70\% \pm 1.12\%$	0.7

(0.107 ± 0.002) across all five architectures, illustrating finding F3 (§5.1): train loss is invisible to a $\geq 100\times$ swing in autoregressive rollout error (0.11% vs. 10.96%).

The full comparison on the multi-grid synthetic data regime—where memorization is unavailable and rule learning is required—is reported in Table 2 and analyzed in §5.5.

The synthetic-multi-grid setting is the headline comparison: per-state memorization is unavailable (4 widths $\times$ 256 synthetic levels per width $= 1024$ distinct levels share one rule set), so the model must *learn the rule that maps state plus action to the next state*. Three observations: **(i)** the NCA with shared weights again wins on BFS rollout ($4.37\% \pm 2.02\%$) at the fewest parameters (3.07M), beating the matched-depth CNN ($5.56\% \pm 3.12\%$) and the per-step NCA ($5.95\% \pm 3.50\%$); the $2.8\times$ parameter advantage over NCA-per-step suggests the shared-weights inductive bias is doing real work, not just compressing capacity; **(ii)** U-Net ($8.33\% \pm 0.97\%$) and ViT ($12.70\% \pm 1.12\%$) underperform here as well, with ViT also failing to fit the train loss (0.084 ± 0.002 vs. 0.069 ± 0.003 for the other four); **(iii)** the seed-to-seed variance on this task is large ($\sigma > 2\%$ on BFS error for all NCA / CNN variants), reflecting the multiple-basin optimization landscape we previously characterized on the closely related VARISLIDE task (F8); the large gap to U-Net and ViT is robust to that variance.

(F1) Pool features are load-bearing. A bare 3×3 convolution can only propagate one cell of context per NCA step. Several PuzzleScript rule patterns reference cells arbitrarily far apart along an axis or anywhere on the level (`[X | ... | Y]`, `[X][Y]`). Without pool features the NCA needs $\Omega(\max(H, W))$ steps just to surface non-local information; with axis-cummax + global pool one step suffices. Across the 19-game SCALING_LARGE preset and a controlled COLLAPSE no-pool ablation, turning pool off costs both convergence speed and final loss, with the no-pool model collapsing into the identity-prediction minimum even at $4\times$ the depth.

(F2) Shared NCA-body weights are the right inductive bias. The PuzzleScript engine has two natural levels of iteration: an inner ordered list of rules applied once each per tick, and an outer `again` loop that re-runs the rule list until the state stops changing. We factor $T = n_\ell \cdot n_r$ accordingly. On a single-game depth sweep ($T \in \{2, 4, 8, 16\}$), shared T=8 reaches $5.5\times$ lower train loss with $7\times$ fewer parameters than the per-step body at the same depth; shared T=2 at $14\times$ fewer parameters than the deepest per-step variant matches it on oracle-action rollouts. Hierarchical $n_r > 1$ sits between these extremes and is the subject of an ongoing ablation (§5.3).

(F3) Train loss decouples from autoregressive rollout error. A model with the lowest train (per-step) loss is not the model with the lowest autoregressive rollout error. Concretely, in the over-capacity regime we observe per-step models with $5\times$ lower train loss but $3-4\times$ higher 50-step rollout error. The mechanism is small but systematic per-step errors compounding destructively under autoregression. Reporting only best-loss hides the rollout-drift cost of going deeper or wider.

5.2 Pool-feature ablation

[Figure: pool-on vs. pool-off on COLLAPSE, both at depths $T \in \{2, 4, 8, 16\}$, reporting train loss + autoregressive rollout error.]

5.3 Hierarchical weight sharing

[Figure: T=16 varying $n_r \in \{1, 2, 4, 8, 16\}$ on MICROBAN multi-grid synth and on the multi-game SCALING_14 preset. Reports change-error and parameter count.]

5.4 Adaptive halting

We compare four halting modes and a fixed-depth baseline on a custom 1-rule VARISLIDE game in which a single `again`-driven slide rule pushes the player up to $d \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ cells per action across grid widths $\{6, 8, 10, 12, 16\}$. Modes: *ponder* (PonderNet ponder loss with learned halt head + KL prior), *uniform* (per-step loss equally weighted across all T steps), *argmax-ST* (straight-through estimator on $\arg\max_k p_k$), and *convergence-ST* (halt at inference when consecutive predictions stop changing, no learned head).

[Figure: per-step error trajectory under each mode + the per-instance halt distribution as a function of slide distance.]

Across these modes we observe two phenomena. First, on tasks where the dynamics fit cleanly in $\sim$1–2 NCA steps (small-grid varislide, COLLAPSE), all halting modes converge near the dynamics-floor within their effective depth and drift past it; convergence-ST recovers near-oracle performance via inference-time stopping. Second, on the multi-grid synthetic regime where memorization is unavailable, *none* of the halting modes reliably learn the underlying rule (§5.5); the halt head at best degenerates to halt-at-k=1 + identity prediction.

5.5 Multi-grid synthetic data ceiling

When we move from a single authored level to the multi-grid synthetic data regime ($\{6, 8, 10, 12, 16\}$ widths $\times$ 64 levels each), train change-error climbs by $1000\times$ ($5 \times 10^{-5} \rightarrow 0.05$–$0.10$), confirming that the model can no longer 1-shot-memorize per-state outcomes. The model attempts to learn the rule but plateaus: 50k updates at d_h=256 ($5\times$ more updates, $2\times$ wider) does not move the needle. Multi-seed verification at $T \in \{8, 16, 32, 64\}$ shows per-seed argmax variance dwarfs the recipe signal and best-loss is essentially identical across all configurations. We discuss candidate fixes in §6.

5.6 Multi-game scaling

Table 3 reports the head-to-head comparison on the SCALING_14 benchmark: 14 PuzzleScript games sharing the rule-slot encoder, with grid sizes from 4×4 to 32×24 and rule counts from a handful (basic Sokoban) to $\sim$30 (`Heroes_of_Sokoban`, `Sokoboros`). Single seed, 20k updates, $d_h = 256$.

The multi-game story is more nuanced than the single-game one. **(i)** Both NCA variants outperform all three non-NCA baselines on autoregressive rollout, with U-Net at $1.55\times$ and ViT at $\sim2\times$ NCA-perstep's BFS error. **(ii)** Within the NCA family, the per-step body wins on BFS rollout (10.46%) while the shared body wins on train loss (8.6×10^{-6}, almost $2\times$ lower than per-step). The shared body's lower train loss combined with its higher rollout error is a textbook instance of F3 (§5.1): per-step capacity helps the model encode the joint distribution of 14 distinct rule sets, while the shared body's compactness pays a small cost on rollout drift. **(iii)** ViT's train loss is now $\sim80\times$ higher than NCA-shared (1.35×10^{-3} vs. 8.6×10^{-6}), confirming the structural mismatch we observed on the synthetic single-game setting (§5.1): unrestricted global attention does not fit grid-rule dynamics well even at modest scale. **(iv)** Per-game breakdowns and held-out cross-game generalization are deferred to the appendix.

Table 3: **Multi-game baselines on SCALING_14.** Single seed at 20k updates. Mean cell-error rate averaged across games and levels. Both NCA variants beat all three non-NCA baselines; within the NCA family, the per-step body wins on BFS rollout while the shared body wins on train loss (parameter-efficient).

Architecture	params (M)	best loss	BFS final-err	A* final-err
NCA (per-step, T=4, n_r=1)	7.37	1.6×10^{-5}	$\mathbf{10.46}\%$	$\mathbf{11.88}\%$
CNN (4-block ResNet)	5.01	1.8×10^{-5}	12.14%	12.94%
NCA (shared, T=4, n_r=T)	**1.96**	$\mathbf{8.6 \times 10^{-6}}$	13.44%	14.64%
U-Net (2 levels)	6.09	1.1×10^{-4}	20.81%	23.59%
ViT (4 layers)	4.01	1.4×10^{-3}	26.29%	26.47%

6 Discussion

What the iterated local update buys. The architecture comparison in §5.1 isolates one question: holding the conditioner constant, does the NCA's iterated-local-update body outperform a non-iterative body? Our findings suggest the answer is yes on tasks where the engine's `again` loop produces non-trivial cascade chains, and roughly a wash on tasks where one tick of dynamics resolves in a single rule firing. The NCA's parameter efficiency is also striking: shared weights with $T \cdot n_\ell$ effective depth use the same parameters as a one-step model.

Memorization as a confound. A key observation in §5.5 is that the single-level / single-grid setting allows the model to learn a per-state lookup table via cross-attention over rich slot representations. "Pool features substitute for depth" is the single-level reading; "the model memorizes per-state outcomes" is the multi-level reading of the same architectural configuration. Both are real, and disentangling them required the multi-grid synthetic-data regime that we describe in §5.5. We recommend the multi-grid synth setup as a default check for future PuzzleScript world-model work: if your model only succeeds at the single-grid setting, you may not be learning the dynamics.

Open: escaping the multi-grid ceiling. Our experiments characterize the ceiling but do not break it. The remaining candidate fixes that have not been ruled out are (a) hard / sparse rule-slot routing so the model cannot smear "fire-once" across all slots, (b) per-cell halt mechanism so cells in their fixed point stop updating, (c) curriculum or replay-buffer training that selectively exposes the model to slide-distance ≥ 2 instances, and (d) auxiliary "is-changing" mask head factoring "where to fire" from "what to write." A successful break here would close the most important remaining gap between authored-data and synthetic-multi-grid performance.

Beyond PuzzleScript. The architecture is not domain-specific. Any deterministic discrete-grid simulator with declarative rules (Conway's Game of Life and its variants, cellular-automata gas dynamics, Asari-style chemical-reaction grids) admits the same modeling pattern: tokenize the rules, encode them via the slot encoder, train an NCA body shared across iterations. We expect the comparison against non-iterative baselines to favor the NCA more strongly as the underlying engine's outer-loop iteration count grows.

Limitations. Our slot encoder is level-independent: the slots depend only on the game's rule tokens, not on the level's spatial state. This is the intended design (one encoded game $\Rightarrow$ many levels) but it also caps what the slots can carry. A version where slots can attend back into the input grid (perceiver-IO style) would be a natural extension. We also restrict to single-step prediction; multi-step training (predict s_{t+k} for $k > 1$) is straightforward to add and not yet explored.

References

[1] Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder. In *ICML Workshop on Automated Machine Learning*, 2021.

[2] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2018.

[3] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[4] David Ha and Jürgen Schmidhuber. World models. In *Advances in Neural Information Processing Systems*, 2018.

[5] Stephen Lavelle. PuzzleScript: an open-source html5 puzzle game engine. `https://www.puzzlescript.net/`, 2013.

[6] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. *arXiv preprint arXiv:2209.00588*, 2022.

[7] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020.

[8] Elias Najarro, Shyam Sudhakaran, Claire Glanois, and Sebastian Risi. Hypernca: Growing developmental networks with neural cellular automata. In *ICLR Workshop on From Cells to Societies: Collective Learning across Scales*, 2022.

[9] Eyvind Niklasson, Alexander Mordvintsev, Ettore Randazzo, and Michael Levin. Self-organising textures. *Distill*, 2021.

[10] Rasmus Berg Palm, Miguel González-Duque, Shyam Sudhakaran, and Sebastian Risi. Variational neural cellular automata. In *International Conference on Learning Representations*, 2022.

[11] Jan Robine, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. Transformer-based world models are happy with 100k interactions. In *International Conference on Learning Representations*, 2023.

[12] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems*, 2015.

A Implementation details

A.1 Model construction

All models are implemented in JAX/Flax with the Adam optimizer. The slot encoder is identical across architectures (see §3). The hidden grid dimension d_h, the cell update structure, and the readout heads are also shared. A single `-architecture` flag selects the spatial-update body (`rule_attn`, `cnn`, `unet`, `vit`).

A.2 Tokenization

Each game's tokenized representation is a sequence of integers from a $\sim$140-symbol vocabulary covering: object names, layer references, rule operators, action types, the 5×5 sprite RGBA values when sprite tokens are enabled, and per-rule kernel separators. A single token sequence per game serves as the encoder input.

A.3 Per-game SCALING_14 breakdown

Figure 1 shows the per-game BFS final cell-error on SCALING_14, averaged over each game's authored levels. Three observations beyond the aggregate numbers in Table 3:

- TRAVELLING_SALESMAN is hardest for every architecture (NCA-perstep best at 38%); the rule set encodes an NP-hard combinatorial problem that no single-step world model is expected to solve perfectly.
- Several games (NEKOPUZZLE, NOTSNAKE, SOKOBAN_BASIC, BLOCKS) are fit to zero error by the NCA family but fail badly for U-Net (10–25%) and ViT (1–33%). The architecture gap is consistent across diverse rule sets, not an averaged-out artifact.

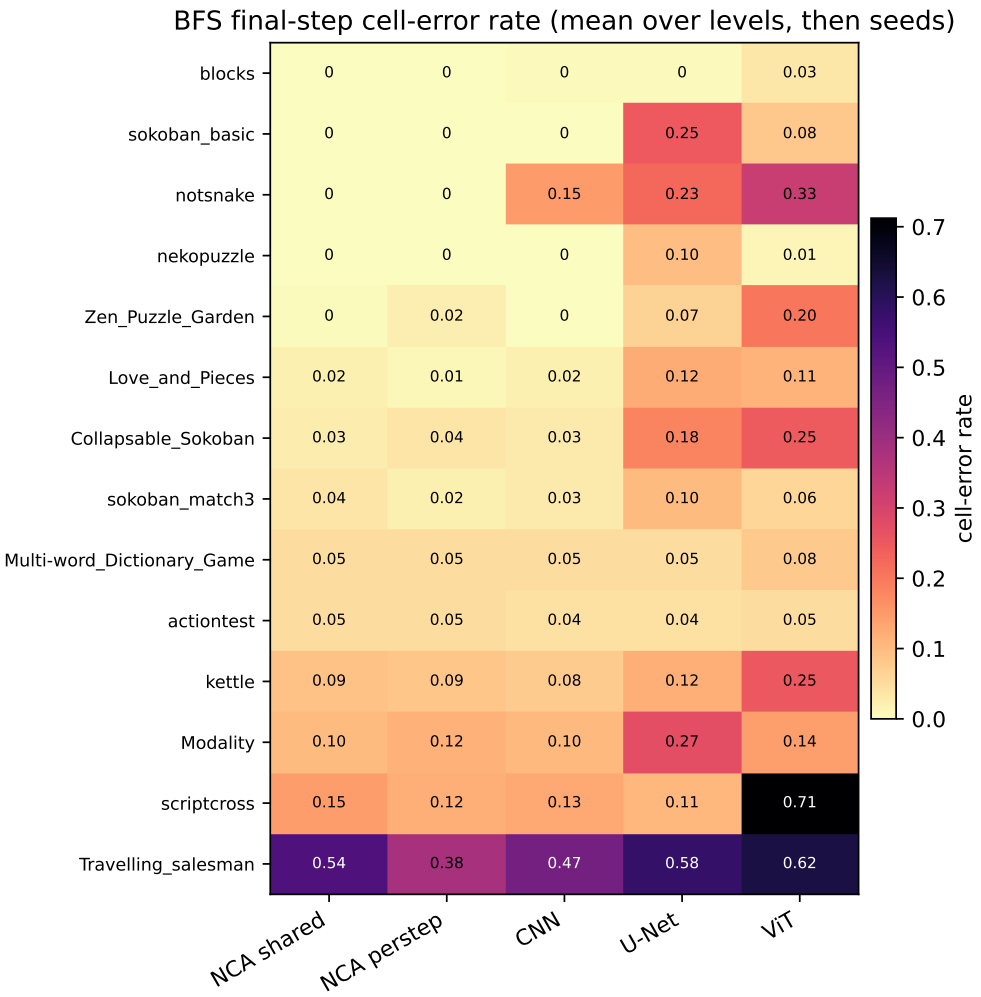

Figure 1: **Per-game BFS final cell-error on SCALING_14.** Rows are games (sorted by NCA-shared error, ascending); columns are architectures. Single seed at 20k updates. Both NCA variants beat the three non-NCA baselines on the vast majority of games; ViT is the only architecture with a catastrophic failure on a small game (SCRIPTCROSS).

- SCRIPTCROSS is a clean ViT-failure case (71% error vs. 11–15% for the rest), illustrating the structural mismatch we discussed in §5.6.

A.4 Reproducibility

The codebase, dataset caches, training launchers, and result aggregators are released alongside the paper. Each run is reproduced end-to-end by re-running its launcher (`nca_wm/scripts/run_*.sh`) on the same hardware (single 24 GB consumer GPU). The full experimental log is in `nca_wm/SCALING_RESULTS.md` and `nca_wm/ARCHITECTURE_REPORT.md` in the released code.